

Sui & Move Bootcamp

Two days. Zero hand-holding. Ship on-chain.

Two-Day Overview

■ Day 1 — The Sui Architect

Thinking in Objects, not Accounts

- Hour 1: The Object Revolution
- Hour 2: Move Fundamentals & Syntax
- Hour 3: Structs, Ownership, & Abilities
- Hour 4: Capability + Hot Potato Patterns
- Hour 5: Deployment & CLI
- Hour 6: Hackathon Reveal

■ Day 2 — The Builder

Bridging Move to the World

- Hour 1: The TypeScript Bridge
- Hour 2: Transaction Blocks (PTBs)
- Hours 3–5: E2E dApp Hackathon
- Hour 6: Vibe Check & Viva Voce

Pre-Flight

Get ready before Day 1.

Zero setup friction on the day means **more time building.**

Install Sui CLI

`brew install sui` or download from docs.sui.io

Install Move Analyzer

VS Code extension for syntax + type checking

Install Sui Wallet

Browser extension — connect to Testnet

Vibe Check

Mint 1 Testnet SUI from faucet. Send it to a peer.

Day 1

The Sui Architect

Thinking in Objects, not Accounts

The Object Revolution

Hour 1 — Theory & Roleplay

■ The Physical Ledger

Each card in the deck is an **Object**.

Some cards have abilities (tradeable). Others don't.

The same rules apply in Move.

■ The Mental Shift

In traditional blockchains, contracts own the data.

In Sui, the user owns the data.

Objects live in wallets — not inside contracts.

Account-Based vs Object-Based Assets

Topic	Account-Based (Ethereum-style)	Object-Based (Sui Move)
Unit of state	Account storage slots	Individually addressable objects
Ownership	Account controls balance + storage	Each object has a direct owner or shared mode
Transfer model	Update balances / storage mappings	Move object ownership from one owner to another
Parallelism	Many txns contend on shared global state	Independent objects can execute in parallel

Who Owns The Asset?

ACCOUNT MODEL (ETH-style)

Alice Account

- ETH balance: 12
- ERC20[USDC]: 100
- NFT[42]: true

Assets are "entries inside account/contract storage".

OBJECT MODEL (SUI)

Object 0xA1: Coin<SUI>	owner = Alice
Object 0xB2: ProfileNFT	owner = Alice
Object 0xC3: ManagerCap	owner = Alice

Assets are first-class objects with their own IDs and owners.

Why Objects Are Composable

You can **wrap objects inside objects** to model nested rights and workflows.

```
public struct Badge has key { id: UID, level: u8 }  
  
public struct Backpack has key {  
    id: UID,  
    badge: Badge, // object composition  
}
```

In account-based models, composition is usually indirect (mappings/IDs).

In Sui, composition is native to the type system.

Object States That Matter

- **Address-owned**: one wallet owns it; fast path is possible.
- **Object-owned**: another object owns it (wrapped/composed state).
- **Shared**: many users can access; ordering/consensus is required.
- **Immutable**: readable by all, mutable by none.

Why this matters: state type determines performance profile, access model, and API design.

Consensus vs Fast Path (And Parallelism)

Owned-object txn (independent objects)

Tx A: use Object X (owned by Alice)

Tx B: use Object Y (owned by Bob)

-> can run in parallel, often without shared-object consensus path

Shared-object txn

Tx A: mutate Shared Pool P

Tx B: mutate Shared Pool P

-> ordering is required, consensus path applies

Rule of thumb: prefer owned-object flows when possible; use shared objects only when coordination is required.

From Sui Concepts To Move Code

Hour 2 — Move Fundamentals & Syntax

- We now map object mental models into Move modules and functions.
- We start with package layout, compiler flow, and project scaffolding.
- Then we step into syntax: module members, types, control flow, and calls.

Move Package Anatomy & Compiler Flow

```
my_first_package/  
  Move.toml      # package manifest  
  sources/  
    hero.move    # modules  
  tests/  
    hero_tests.move # optional
```

Compiler view: `Move.toml` -> `modules` -> `bytecode` -> `publish to package address`.

Scaffold and start:

```
sui move new my_first_package  
cd my_first_package  
sui move build  
sui move test
```

Packages, Modules, and Addresses

- A published package has an on-chain address (for example: `0xabc...`).
- Modules live inside that package: `0xabc...::hero`.
- Functions/types are addressed from that module:
 - `0xabc...::hero::mint`
 - `0xabc...::hero::Hero`

Mental model: **package::module::member**.

Abstract Module Skeleton

```
module demo::hero {  
  use std::string::String;  
  use sui::object::{Self, UID};  
  use sui::tx_context::TxContext;  
  
  const ENotAdmin: u64 = 0;  
  
  public struct Hero has key, store {  
    id: UID,  
    name: String,  
    level: u64,  
  }  
  
  public fun mint(name: String, ctx: &mut TxContext): Hero {  
    Hero { id: object::new(ctx), name, level: 1 }  
  }  
  
  fun level_up(hero: &mut Hero) {  
    hero.level = hero.level + 1;  
  }  
}
```

Don't worry about full syntax mastery yet; this is a map, not memorization.

Data Types, Variables, and Safety

- Choose types by intent: `u8/u64/u128`, `bool`, `address`, vectors, structs.
- Local bindings use `let`; module constants use `const`.
- Casting is explicit, for example `x as u64` when needed.
- Overflows are arithmetic errors; prefer checked design and realistic bounds.

Ask every time: *what kind of value is this, and what range can it reach?*

Strings (Move 2024 Quick Note)

- Prefer `std::string::String` for text data.
- Common pattern: `b"Hero".to_string()`.
- Avoid older ad-hoc byte handling for user-facing text unless explicitly needed.

One-slide takeaway: use modern `String` APIs by default.

Control Flow Expressions (Quick Note)

- `if / else` for branching.
- `while` for loops.
- `return` for early exits.
- `assert!(condition, ECode)` for invariant checks.

Keep branching simple and push complexity into helper functions.

Function Calls, References, and Ownership

```
public fun upgrade_hero(  
    hero: &mut Hero,  
    points: u64,  
    ctx: &mut TxContext  
) {  
    let _sender = tx_context::sender(ctx);  
    hero.level = hero.level + points;  
}
```

- Pass by `&` / `&mut` when you only need to read or mutate in-place.
- Pass by value when ownership should move.
- We include objects as params so function access is explicit and verifiable.

Important Rule: No Dangling Resources

Yes, this is the right point to teach it.

If a function takes an object **by value**, it must be:

- returned, or
- transferred/stored, or
- explicitly destroyed (if allowed).

Only references (`&` / `&mut`) can go out of scope without moving ownership.

Define, Create, and Transfer (One Snippet)

```
public struct Hero has key, store {  
    id: UID,  
    name: String,  
}  
  
public fun mint_and_send(name: String, to: address, ctx: &mut TxContext) {  
    let hero = Hero { id: object::new(ctx), name };  
    transfer::public_transfer(hero, to);  
}
```

key matters because on-chain objects need identity (**UID**) and object semantics.

Unpacking a Struct

Unpacking means destructuring fields out of a value.

```
public fun burn_hero(hero: Hero) {  
    let Hero { id, name: _ } = hero;  
    object::delete(id);  
}
```

Use it when you need to consume a resource and handle its fields directly.

Structs & Abilities

Hour 3 — Ownership Semantics

■ Ability: **key**

Defines an on-chain object with identity (**UID**).

Correct:

- `Hero has key { id: UID, ... }`
- `ManagerCap has key { id: UID }`

Wrong:

- `Broken has key { count: u64 }` (missing `id: UID`)

■ Ability: **store**

Allows values to be stored in global/object state.

Correct:

- `TradeableItem has key, store { ... }`
- `Metadata has store { name: String }`

Wrong:

- putting non-**store** value into persistent storage.

More Abilities

■ Ability: `copy`

Allows duplication by value.

Correct:

- numeric primitives (`u64`) are copyable
- small config structs with `copy` semantics

Wrong:

- `Coin<SUI>` with `copy` (would violate resource guarantees)

■ Ability: `drop`

Allows values to be discarded.

Correct:

- ephemeral helper structs with `drop`
- event payload helper types (when appropriate)

Wrong:

- loan receipt that should never be silently forgotten

Soulbound vs Transferable

```
public struct SoulboundID has key {  
    id: UID,  
}  
  
public struct TradeableItem has key, store {  
    id: UID,  
    name: String,  
}
```

- Omit `store` when you want stronger non-transfer behavior patterns.
- Use `key, store` for assets meant for open transfer/marketplaces.

Ability Patterns In Real Systems

- **OTW (One-Time Witness):** enforce one-time initialization patterns.
- **Capabilities:** make authority an object (`AdminCap` , `ManagerCap`).
- **Hot Potato:** zero-ability receipts that must be consumed.
- **Soulbound:** constrained transfer semantics by ability design choices.

Abilities are your first security layer, not just syntax decoration.

The Capability Pattern

Hour 4 — Authority

Digital authority is a physical asset you can hold, transfer, or burn.

- A `ManagerCap` is just an object in your wallet
- Functions gate themselves by requiring it as a parameter
- Lose the cap — lose the authority. Transfer it — transfer the power.

```
public fun admin_only(_cap: &ManagerCap, /* ... */) {  
    // only callable if caller holds ManagerCap  
}
```

Lunch Break

One hour. Come back hungry to build.

Hot Potatoes

Hour 4 — The Anti-Bug Pattern

A struct with **zero abilities** cannot be dropped or stored.

The compiler forces you to consume it.

This is a feature, not a bug.

Step 1 — Create

`take_loan` returns a `LoanReceipt` with no abilities.

Step 2 — Compiler Error

Try to end the transaction without consuming it. It fails.

Step 3 — The Fix

Write `repay_loan` that takes the receipt and destroys it.

The Insight

Impossible to forget repayment. The `type system` is the guardrail.

Deployment & CLI Mastery

Hour 5

Publish to Testnet and prove it exists.

```
# Publish the package
sui client publish --gas-budget 100000000

# Call a function on-chain
sui client call \
  --package <PACKAGE_ID> \
  --module hero \
  --function mint \
  --gas-budget 10000000
```

Then verify the object ID on suiexplorer.com.

Hackathon Reveal

Hour 6 — Team Forming

■ The Challenge

Build a **Peer-to-Peer Rental System**.
What objects exist? Who owns them?

■ Deliverable

A 1-page **Architecture Map** — signed off by the instructor before Day 2.

■ Object Diagram

Teams use AI to draft an Object Relationship Diagram before writing any code.

■ The Rule

No code without a diagram. Design before you build.

Day 2

The Builder

Bridging Move to the World

The TypeScript Bridge

Hour 1 — Modeling

■ Connect to Sui SDK

Wire a React frontend to `@mysten/sui`.

Fetch all objects owned by an address and render their fields.

```
const objects = await client
  .getOwnedObjects({ owner: address });
```

■ Display Object Fields

Read struct fields from on-chain objects and display them in the UI.

The blockchain is your database. The SDK is your ORM.

Transaction Blocks (PTBs)

Hour 2

■ Pure Arguments

Primitives passed directly into a transaction.

`string`, `number`, `bool` — encoded inline.

```
tx.pure.string("Hello")  
tx.pure.u64(42n)
```

■ Object Arguments

On-chain objects referenced by ID.

The wallet resolves ownership and signs.

```
tx.object("0xABC...")  
tx.object(tx.gas)
```

A Button That Pays

The goal of Hour 2 is one working button:

1. User clicks **"Rent"**
2. PTB is constructed with object + coin args
3. Wallet popup appears — user signs
4. Transaction confirmed on-chain

Everything else in Web3 is a variation of this flow.

Lunch Break

Last one. Hours 3–5 are the build sprint.

The Decentralized Rental Hub

Hours 3–5 — E2E Hackathon

■ Feature A

Owner mints a `RentalProperty` object.
Stored in their wallet. They control it.

■ Feature B

Tenant pays SUI and receives a `KeyCard` capability.
The card grants access. It can expire or be returned.

■ Feature C

A **Hot Potato** ensures the tenant cannot leave without paying the cleaning fee.
The type system enforces checkout.

Build Guidelines

What to use for each feature:

Feature	Pattern
Mint property	<code>key</code> + <code>store</code> struct, <code>transfer::transfer</code>
Issue key card	Capability pattern, time-bound field
Cleaning fee	Hot potato — zero-ability receipt
Ownership transfer	<code>transfer::transfer</code> vs <code>transfer::share_object</code>

Know why you chose each one — you will be asked.

Vibe Check

Hour 6 — Demo & Viva Voce

Show the UI. Explain the code. Pass.

The Pitch

Live demo: UI + wallet working end-to-end on Testnet.

The Viva Voce

One student per team answers: "Show me where ownership changes in your Move code."

The Question

"Why did you use `transfer` instead of `share_object`?"

How to Pass

Explain the Object Flow. That's it.

If You Can Explain the Object Flow

You understand Sui.